ShopLaravel

Application E-Commerce Complète

Rapport Technique

Framework Laravel 11 · PHP 8.2 · MySQL

Frontend : Blade · Tailwind CSS · Alpine.js

Environnement : WAMP / XAMPP

Démonstration : 23 Avril 2026

ÉQUIPE DE DÉVELOPPEMENT

Membre	Rôle	Modules
Serine Khelifi ★ Chef de Projet	Développeur Auth	Authentification · Profil utilisateur
Belhassen Mechichi	Lead Développeur	Architecture · Middleware Admin · Rôles
Aziz Younsi	Développeur Produit	CRUD Produits · Upload images · Catégories
Amine Ben Mime	Développeur Commerce	Panier · Commandes · Checkout

ESSEC · Programmation Web 2 · DS2 · Année Universitaire 2025–2026

1. Description du Projet

1.1 Contexte et Objectifs Pédagogiques

Dans le cadre du module Programmation Web 2 (DS2 — ESSEC, 2ème année E-Business, promotion 2025–2026), notre équipe de quatre membres a développé ShopLaravel, une application e-commerce complète, fonctionnelle et sécurisée basée sur le framework Laravel 11. Ce projet démontre notre maîtrise des technologies web modernes à travers la conception d'une plateforme de vente en ligne multi-utilisateurs.

Objectifs pédagogiques couverts :

- Maîtriser un framework PHP moderne (Laravel 11) dans un contexte professionnel
- Concevoir et implémenter une architecture MVC complète et organisée
- Manipuler une base de données relationnelle avec l'ORM Eloquent
- Implémenter une logique métier e-commerce réaliste (panier, commandes, évaluations)
- Appliquer les bonnes pratiques de sécurité web (CSRF, XSS, injections SQL)
- Développer un projet structuré en équipe avec répartition claire des tâches

1.2 Fonctionnalités Obligatoires Implémentées

Module	Fonctionnalités	Statut
Authentification	Inscription · Connexion · Déconnexion · Vérification email Réinitialisation mot de passe · Gestion du profil	✓ Complet
Produits (CRUD)	Ajout · Modification · Suppression · Upload image principale + galerie multiple · Auto-génération du slug	✓ Complet
Catalogue	Affichage public · Recherche par mot-clé · Filtrage par catégorie Tri (prix, date, popularité)	✓ Complet
Panier	Ajout · Modification de quantité · Suppression d'articles Vidage complet · Persistance côté serveur	✓ Complet
Commandes	Checkout complet · Historique des commandes Statuts : En attente → Validée → Expédiée → Livrée → Annulée	✓ Complet
Évaluations	Notation 1 à 5 étoiles · Commentaire texte Calcul note moyenne · Suppression de ses propres avis	✓ Complet

1.3 Fonctionnalités Bonus

Fonctionnalité	Niveau	Description
Gestion des rôles Admin/User	Niveau 3 (+3 pts)	Panel admin complet avec middleware de protection, gestion des utilisateurs et des rôles
Chat entre utilisateurs	Niveau 3 (+3 pts)	Messagerie privée entre utilisateurs avec historique
Multi-catégories (Many-to-Many)	Niveau 2 (+2 pts)	Un produit peut appartenir à plusieurs catégories
Filtres avancés (Scopes Eloquent)	Niveau 2 (+1 pt)	Scopes chainables : active, featured, inStock, search...
Upload d'images (Storage)	Niveau 1 (+1 pt)	Stockage dans storage/app/public avec lien symbolique
Wishlist	Niveau 1 (+1 pt)	Liste de favoris Many-to-Many avec toggle AJAX
Pagination personnalisée	Niveau 1 (+1 pt)	Pagination Tailwind CSS sur le catalogue produits

Note : Conformément au cahier des charges, seul le bonus de niveau le plus élevé est comptabilisé. Notre projet implémente des fonctionnalités de Niveau 3 (Gestion des rôles + Chat), soit +3 points bonus.

1.4 Stack Technique

Technologie	Version	Rôle dans le projet
Laravel	11.x	Framework PHP — MVC, Eloquent ORM, Migrations, Blade, Middleware
PHP	>= 8.2	Langage d'exécution côté serveur
MySQL	8.x	Système de gestion de base de données relationnelle
Tailwind CSS	CDN v3	Framework CSS utilitaire — design responsive et moderne
Alpine.js	CDN	Micro-framework JS pour l'interactivité légère (dropdowns, toggles)
Chart.js	CDN	Graphiques dans le panneau administrateur (revenus mensuels)
SweetAlert2	CDN	Notifications et alertes flash stylisées
Lucide Icons	CDN	Bibliothèque d'icônes SVG modernes
Faker (PHP)	via Composer	Génération de données de test réalistes pour les seeders
WAMP / XAMPP	Dernière	Environnement de développement local (Apache + MySQL + PHP)

2. Architecture du Projet (MVC)

2.1 Patron Architectural MVC

ShopLaravel respecte strictement le patron architectural MVC (Modèle – Vue – Contrôleur) de Laravel. Ce patron garantit une séparation nette des responsabilités, facilitant la maintenabilité, les tests et l'évolution du code.

Couche	Rôle	Exemples dans ShopLaravel
Modèle (Model)	Représente les données, les règles métier et les interactions avec la BDD via Eloquent ORM	User, Product, Category, Order, OrderItem, Review, Wishlist, Message, OrderIte
Vue (View)	Affiche les données. Templates Blade avec héritage de layouts et composants réutilisables	layouts/app.blade.php · layouts/admin.blade.php · components/product-card.
Contrôleur (Controller)	Traite les requêtes HTTP, coordonne les modèles et renvoie une réponse	ProductController · CartController · OrderController · Admin/Dashboa
Routes	Définit le mapping URL → Contrôleur, organisées en 4 groupes avec middleware	Public, Guest, Authentifié, Admin
Middleware	Filtre les requêtes avant qu'elles atteignent le contrôleur (auth, guest)	EnsureUserIsAdmin (abort 403)
Form Requests	Validation découplée des données de formulaire avec messages d'erreur en français	StoreProductRequest · UpdateProductRequest · StoreOrderRequest

2.2 Structure des Dossiers

Chemin	Contenu et Description
app/Http/Controllers/Auth/	LoginController · RegisterController · ForgotPasswordController · ResetPasswordController
app/Http/Controllers/Admin/	DashboardController · AdminProductController · AdminUserController · AdminOrderController
app/Http/Controllers/	ProductController · CartController · OrderController · ReviewController · WishlistController · MessageC
app/Http/Middleware/	EnsureUserIsAdmin.php — vérifie le rôle admin, sinon abort(403)
app/Http/Requests/	StoreProductRequest · UpdateProductRequest · StoreOrderRequest
app/Models/	User · Product · Category · Order · OrderItem · Review · Wishlist · Message
database/migrations/	4 fichiers de migration — création structurée de toutes les tables
database/seeders/	DatabaseSeeder.php — 50 produits, 8 catégories, 7+ utilisateurs (Faker)
resources/views/layouts/	app.blade.php (layout principal Tailwind) · admin.blade.php (sidebar sombre)
resources/views/	auth/ · products/ · cart/ · orders/ · reviews/ · profile/ · wishlist/ · messages/ · admin/ · errors/
routes/web.php	Routes organisées : Public, Guest, Authentifié (auth), Admin (admin)
storage/app/public/	Images uploadées — accessibles via lien symbolique
.env	Configuration sensible (BDD, clé application, mail, timezone: Africa/Tunis)

2.3 Système de Routage

Groupe	Middleware	Routes principales
Public	Aucun	GET / · GET /catalogue · GET /produits/{slug} · GET /categories/{slug}
Guest	guest	GET/POST /connexion · GET/POST /inscription · /mot-de-passe/oublie · /mot-de-passe/reset
Authentifié	auth	Profil · Mes produits · Panier · Commandes · Wishlist · Messages · Avis
Admin	auth + admin	/admin · /admin/produits (resource) · /admin/categories · /admin/utilisateurs · /admin/commande

2.4 Middleware et Sécurité des Routes

Le middleware EnsureUserIsAdmin protège toutes les routes du panneau d'administration. Il vérifie que l'utilisateur connecté possède le rôle admin via la méthode isAdmin() du modèle User. En cas d'accès non autorisé, la réponse HTTP 403 Forbidden est retournée.

3. Base de Données

3.1 Schéma Relationnel

La base de données shop_ds2 contient 9 tables interconnectées par des clés étrangères :

```
users ■■■■ products ■■■ category_product ■■■ categories ■ ■■■ orders ■■■■
order_items ■■■ reviews (liées à products via product_id) ■■■ wishlists (liées à
products via product_id) ■■■ messages (sender_id → users · receiver_id → users)
```

3.2 Description des Tables Principales

Table : users

Colonne	Type	Description
id	BIGINT UNSIGNED PK	Clé primaire auto-incrémentée
name	VARCHAR(191)	Nom complet de l'utilisateur
email	VARCHAR(191) UNIQUE	Adresse email — identifiant de connexion
password	VARCHAR(255)	Mot de passe hashé automatiquement (bcrypt)
role	ENUM('user','admin')	Rôle de l'utilisateur — 'user' par défaut
phone	VARCHAR(20) NULL	Numéro de téléphone (optionnel)
address	TEXT NULL	Adresse de livraison par défaut (optionnel)
email_verified_at	TIMESTAMP NULL	Date de vérification de l'email

Table : products

Colonne	Type	Description
id	BIGINT UNSIGNED PK	Clé primaire
user_id	BIGINT FK → users	Vendeur/propriétaire du produit
title	VARCHAR(191)	Titre du produit
slug	VARCHAR(191) UNIQUE	Identifiant URL (généré automatiquement depuis le titre)
description	TEXT	Description détaillée du produit
price	DECIMAL(10,2)	Prix de vente actuel
old_price	DECIMAL(10,2) NULL	Ancien prix pour afficher le pourcentage de réduction
stock	INT DEFAULT 0	Quantité disponible en stock
image	VARCHAR NULL	Image principale stockée dans storage/
images	JSON NULL	Galerie d'images additionnelles (tableau JSON)
is_active	BOOLEAN DEFAULT true	Produit visible dans le catalogue ou masqué
is_featured	BOOLEAN DEFAULT false	Mis en avant sur la page d'accueil

Tables : order_items · reviews · wishlists · messages · category_product

Table	Colonnes	Description
order_items	order_id / product_id / quantity / price	Lignes de commande avec snapshot du prix unitaire

Table	Colonnes	Description
reviews	user_id / product_id / rating / comment	Évaluations produit de 1 à 5 étoiles avec commentaire
wishlists	user_id / product_id	Liste de favoris Many-to-Many
messages	sender_id / receiver_id / body / read_at	Messagerie privée — NULL si non lu
category_product	category_id / product_id	Table pivot Many-to-Many catégories ↔ produits

3.3 Relations Eloquent ORM

Modèle Source	Type de relation	Description
User → Product	hasMany	Un utilisateur peut publier plusieurs produits
User → Order	hasMany	Un utilisateur peut passer plusieurs commandes
User → Review	hasMany	Un utilisateur peut écrire plusieurs avis
User ↔ Product (wishlist)	belongsToMany	Favoris — Many-to-Many via table wishlists
User → Message (envoyés)	hasMany (sender_id)	Messages envoyés par l'utilisateur
User → Message (reçus)	hasMany (receiver_id)	Messages reçus par l'utilisateur
Product → User	belongsTo	Un produit appartient à un vendeur
Product ↔ Category	belongsToMany	Many-to-Many via table pivot category_product
Product → Review	hasMany	Un produit peut avoir plusieurs évaluations
Order → User	belongsTo	Une commande appartient à un utilisateur
Order → OrderItem	hasMany	Une commande contient plusieurs lignes d'articles
Category ↔ Product	belongsToMany	Une catégorie regroupe plusieurs produits
Category → Category	belongsTo (parent)	Auto-référence pour les sous-catégories

4. Diagrammes UML

4.1 Diagramme de Classes

Le diagramme de classes représente l'ensemble des modèles Eloquent du projet ShopLaravel, leurs attributs, méthodes et relations. Chaque classe correspond directement à une table dans la base de données shop_ds2. Les multiplicités indiquent le type de chaque association (1..*, 0..*, etc.).

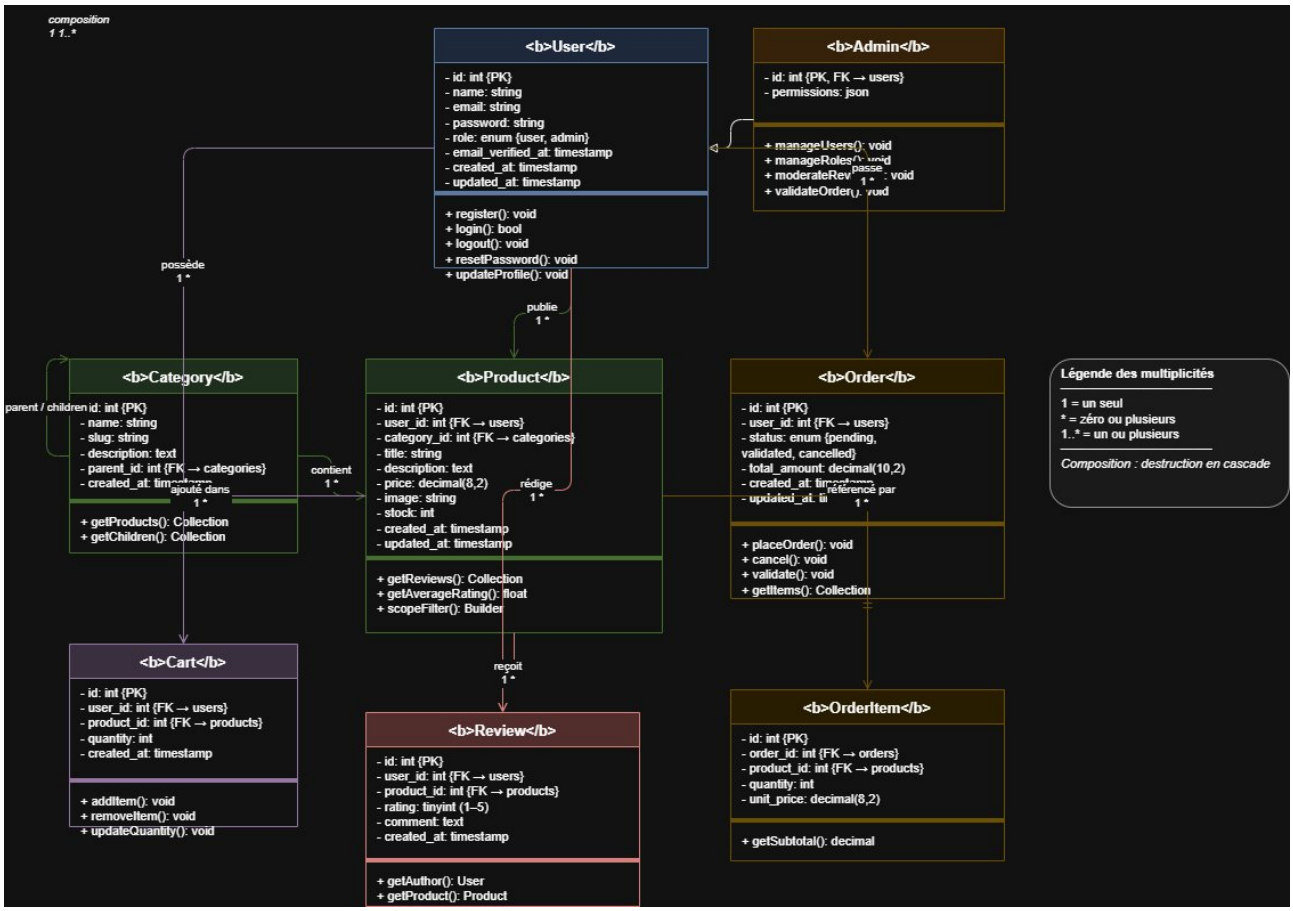

Figure 1 — Diagramme de Classes UML — ShopLaravel

Relation	Type	Exemple dans le projet
User → Product	Composition (1 → 1..*)	Un utilisateur possède ses produits — suppression en cascade
User → Order	Association (1 → *)	Un utilisateur passe plusieurs commandes
Product ↔ Category	Association Many-to-Many	Un produit appartient à plusieurs catégories
Order → OrderItem	Composition (1 → 1..*)	Une commande contient ses lignes d'articles
Product → Review	Association (1 → *)	Un produit reçoit plusieurs évaluations
User ↔ Product (Cart)	Association (* → *)	Un utilisateur ajoute plusieurs produits au panier
User ↔ Admin	Héritage / Extension	Admin étend User avec des permissions supplémentaires

4.2 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation illustre les interactions entre les trois acteurs du système (Visiteur, Utilisateur Authentifié, Administrateur) et l'ensemble des fonctionnalités de la plateforme. Les relations «include» modélisent des dépendances obligatoires et les relations «extend» modélisent des comportements optionnels.

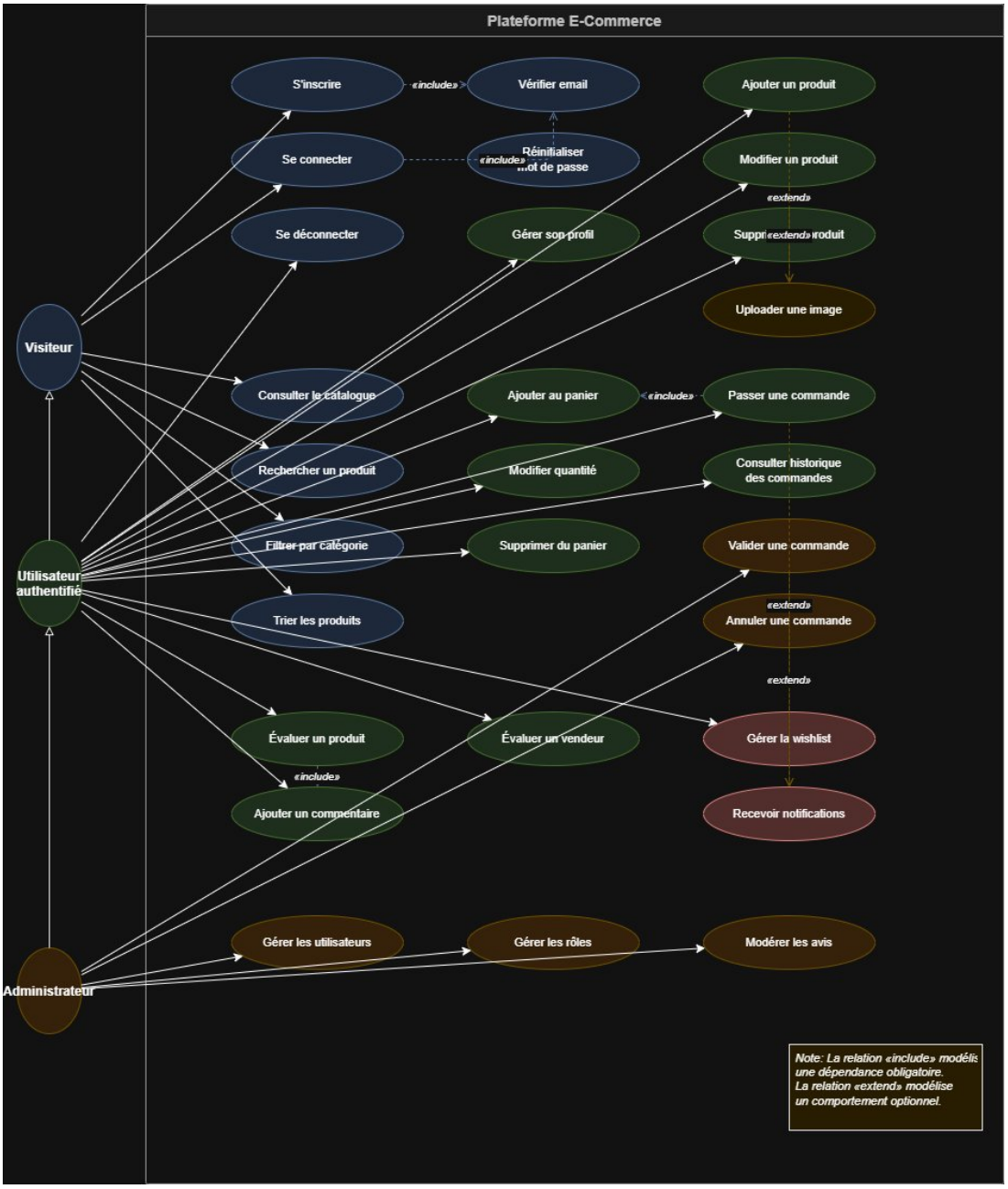

Figure 2 — Diagramme de Cas d'Utilisation UML — Plateforme E-Commerce

Acteur	Cas d'utilisation accessibles
Visiteur	Consulter le catalogue · Rechercher un produit · Filtrer par catégorie · Trier les produits · Voir le détail d'un pro
Utilisateur Authentifié	Tout du Visiteur + Se déconnecter · Gérer son profil · CRUD ses produits · Uploader images · Gérer le panier
Administrateur	Tout de l'Utilisateur + Gérer tous les utilisateurs · Gérer les rôles · Modérer les avis · Gérer toutes les commar

5. Sécurité Implémentée

La sécurité est une priorité dans ShopLaravel. Toutes les vulnérabilités classiques du web ont été prises en compte et traitées conformément aux bonnes pratiques Laravel :

Vulnérabilité	Solution Implémentée	Localisation
CSRF	Directive @csrf sur TOUS les formulaires POST/PUT/DELETE	Toutes les vues Blade
XSS	Utilisation exclusive de {{ $var }} (échappement automatique HTML)	Toutes les vues Blade
Injection SQL	Eloquent ORM avec prepared statements automatiques	Tous les modèles et contrôleurs
Mass Assignment	Propriété $fillable définie explicitement sur chaque modèle	app/Models/*.php
Accès routes admin	Middleware EnsureUserIsAdmin — vérifie isAdmin(), sinon abort(403)	app/Http/Middleware/
Modification ressources	Vérification du propriétaire (user_id === auth()->id())	ProductController · ReviewController
Mots de passe	Hash automatique via 'password' => 'hashed' dans $casts	app/Models/User.php
Données invalides	FormRequests avec règles de validation strictes en français	app/Http/Requests/*.php
Routes sans auth	Middleware 'auth' sur toutes les routes nécessitant une session	routes/web.php
Email non vérifié	Interface MustVerifyEmail implémentée sur le modèle User	app/Models/User.php

6. Fonctionnalités Bonus — Description Technique

6.1 Gestion des Rôles Admin/User (Niveau 3 — +3 pts)

Un système de rôles complet a été implémenté avec deux niveaux d'accès : user (rôle par défaut à l'inscription) et admin. La colonne role du modèle User est un ENUM. La méthode helper isAdmin(): bool simplifie les vérifications. Le middleware EnsureUserIsAdmin est appliqué sur le groupe de routes /admin et retourne une erreur 403 si le rôle n'est pas admin.

Le panneau d'administration comprend :

- Tableau de bord avec graphiques Chart.js (revenus mensuels)
- Gestion complète des produits
- Gestion des utilisateurs (modification des rôles, suppression)
- Gestion des commandes (changement de statuts)
- Gestion des catégories

6.2 Chat entre Utilisateurs (Niveau 3 — +3 pts)

Un système de messagerie privée permet aux utilisateurs de communiquer entre eux. La table messages utilise deux clés étrangères référençant la même table users : sender_id (expéditeur) et receiver_id (destinataire). Le MessageController gère l'affichage de la liste des conversations, l'affichage d'une conversation spécifique et l'envoi de messages. La colonne read_at permet de marquer les messages comme lus.

6.3 Multi-Catégories Many-to-Many (Niveau 2 — +2 pts)

Un produit peut appartenir à plusieurs catégories simultanément grâce à une relation Many-to-Many gérée par la table pivot category_product (category_id, product_id). Dans Eloquent : Product::belongsToMany(Category::class) et Category::belongsToMany(Product::class). Les catégories supportent aussi une hiérarchie parent/enfant via le champ parent_id (auto-référence).

6.4 Filtres Avancés avec Scopes Eloquent (Niveau 2 — +1 pt)

Scope	Description
scopeActive($q)	Filtre les produits avec is_active = true
scopeFeatured($q)	Filtre les produits mis en avant (is_featured = true)
scopeInStock($q)	Filtre les produits avec stock > 0
scopeSearch($q, $term)	Recherche LIKE dans le titre et la description
scopeByCategory($q, $id)	Filtre par catégorie via la relation Many-to-Many
scopePriceBetween($q, $min, $max)	Filtre par fourchette de prix
scopeSortBy($q, $sort)	Tri par prix asc/desc, date, popularité, note moyenne

6.5 Upload d'Images, Wishlist et Pagination (Niveau 1)

Upload d'images

Les fichiers image sont stockés dans storage/app/public/ via le disk Laravel public. Un lien symbolique créé par php artisan storage:link les rend accessibles publiquement. Les produits supportent une image principale et une galerie JSON d'images additionnelles.

Wishlist

Chaque utilisateur connecté peut ajouter ou retirer des produits de sa liste de favoris via un bouton toggle. La relation est Many-to-Many via la table wishlists. La méthode hasInWishlist(Product $p) du modèle User vérifie rapidement si un produit est dans les favoris.

Pagination

Le catalogue produits utilise la pagination native de Laravel avec la méthode ->paginate(12). Le design des liens de pagination est personnalisé avec Tailwind CSS.

7. Guide d'Installation

7.1 Prérequis

Logiciel	Version	Rôle
PHP	>= 8.2	Langage d'exécution de Laravel
Composer	2.x	Gestionnaire de dépendances PHP
MySQL / MariaDB	>= 5.7	Base de données (inclus dans WAMP/XAMPP)
WAMP / XAMPP	Dernière version	Environnement de développement local (Apache + MySQL)
Navigateur	Tout navigateur moderne	Accès à l'application et à phpMyAdmin

7.2 Étapes d'Installation

Étape 1 — Créer un nouveau projet Laravel

```
composer create-project laravel/laravel shop-ds2 cd shop-ds2
```

Étape 2 — Copier les fichiers du projet

```
Copier depuis le ZIP : app/ database/ resources/ routes/ bootstrap/app.php Vers :
C:\wamp64\www\shop-ds2\ (écraser les fichiers existants)
```

Étape 3 — Créer la base de données dans phpMyAdmin

```
Ouvrir http://localhost/phpmyadmin Nouvelle base de données : shop_ds2 Interclassement :
utf8mb4_unicode_ci Cliquer Créer
```

Étape 4 — Configurer le fichier .env

```
copy .env.example .env Modifier : APP_NAME="ShopLaravel" DB_CONNECTION=mysql
DB_HOST=127.0.0.1 DB_PORT=3306 DB_DATABASE=shop_ds2 DB_USERNAME=root DB_PASSWORD= (laisser
vide sous WAMP par défaut)
```

Étape 5 — Corriger la limite de clé MySQL (si MySQL < 5.7.7)

```
Dans app/Providers/AppServiceProvider.php, méthode boot() : use
Illuminate\Support\Facades\Schema; Schema::defaultStringLength(191);
```

Étape 6 — Générer la clé d'application

```
php artisan key:generate
```

Étape 7 — Exécuter les migrations et les seeders

```
php artisan migrate:fresh --seed (Crée les 9 tables + 50 produits, 8 catégories, 7+
utilisateurs)
```

Étape 8 — Créer le lien symbolique pour le stockage

```
php artisan storage:link (Permet d'afficher les images uploadées depuis le navigateur)
```

Étape 9 — Lancer le serveur de développement

```
php artisan serve Accéder à : http://127.0.0.1:8000
```

7.3 Comptes de Test

Rôle	Email	Mot de passe	Accès
Administrateur	admin@shop.com	password	Panel admin complet + toutes les fonctionnalités
Utilisateur	aziz@shop.com	password	Fonctionnalités utilisateur standard
Utilisateur	serine@shop.com	password	Fonctionnalités utilisateur standard
Utilisateur	younsi@shop.com	password	Fonctionnalités utilisateur standard
Utilisateur	amine@shop.com	password	Fonctionnalités utilisateur standard

Conclusion

ShopLaravel est une application e-commerce complète développée avec Laravel 11, respectant toutes les contraintes du cahier des charges DS2. Le projet implémente l'ensemble des fonctionnalités obligatoires (authentification, CRUD produits, catalogue, panier, commandes, évaluations) ainsi que des fonctionnalités bonus de niveau 3 (gestion des rôles Admin/User, chat entre utilisateurs).

L'architecture MVC est strictement respectée, la sécurité est assurée contre les principales vulnérabilités web, et le code est structuré et documenté.

ShopLaravel · DS2 Programmation Web 2 ESSEC 2ème Année E-Business · 2025–2026 Serine Khelifi (Chef de Projet) · Belhassen Mechichi · Aziz Younsi · Amine Ben Mime